Внутреннее устройство рантайма Go

Горутины • G/M/P • планировщик • кража работы • GOMAXPROCS
Сетевой опросчик • системные вызовы • рост стека • цена горутины

Главный вопрос

На машине 8 CPU, а приложение создает 100 000 горутин. Кто их выполняет?

Почему Go не создает 100 000 потоков ОС?

Что происходит, когда горутина ждет сеть, mutex или системный вызов?

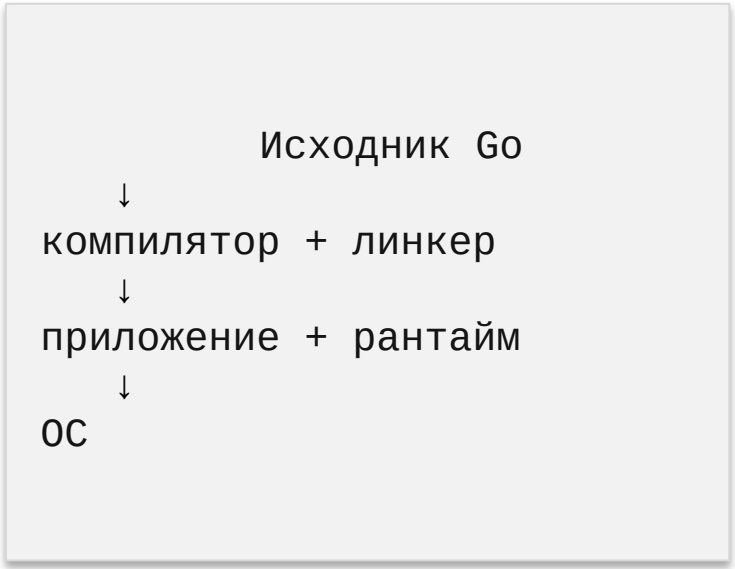
Рантайм Go распределяет множество горутин между ограниченным числом потоков ОС.

Рантайм Go — часть исполняемого файла

Go создает нативный исполняемый файл, но это не «только машинный код».

Рантайм содержит планировщик, GC, аллокатор, стеки, таймеры, сетевой опросчик и служебные метаданные.

В отличие от JVM рантайм Go входит в исполняемый файл приложения.



Горутина: точная модель

Упрощение: горутина = «легкий поток ОС».

Точнее: горутина — независимый контекст выполнения, которым управляет планировщик рантайма Go.

У нее собственный стек и состояние, необходимое для планирования.

`go f()` делает горутину готовой; когда ей достанется CPU, решает планировщик.

G / M / P — три сущности планировщика

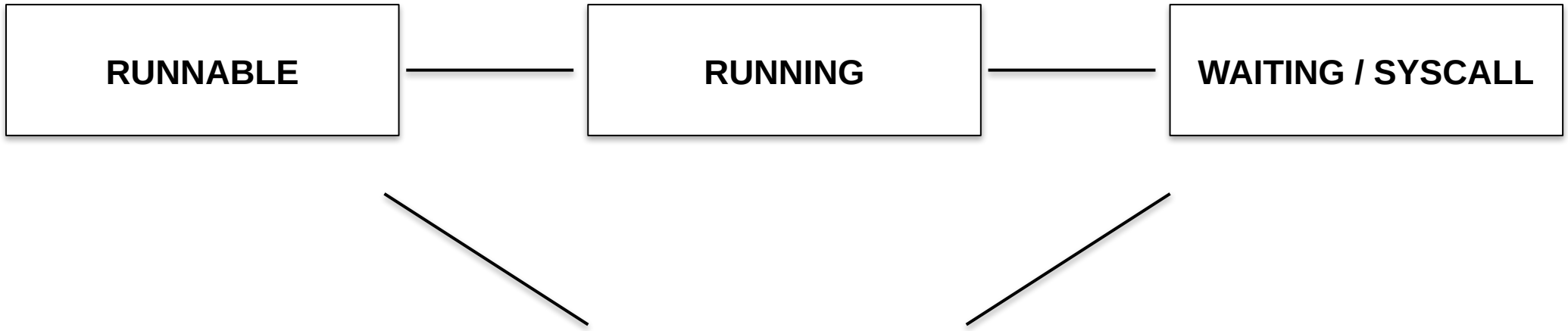


G хранит состояние конкретной горутины.

M исполняет инструкции как поток ОС.

P нужен потоку M для Go-кода и хранит собственные ресурсы рантайма.

Жизненный цикл G



Большинство из 100 000 горутин не исполняются: они ждут сеть, таймер, канал, mutex или другой ресурс.

M — настоящий поток ОС

Поток M создается рантаймом, но планируется ядром ОС.

Именно M физически исполняет машинные инструкции.

Число M может превышать GOMAXPROCS: некоторые потоки блокируются в системных вызовах или cgo.

Планировщик Go выбирает G для M, а планировщик ОС выделяет CPU потоку M.

Зачем вообще нужен P?

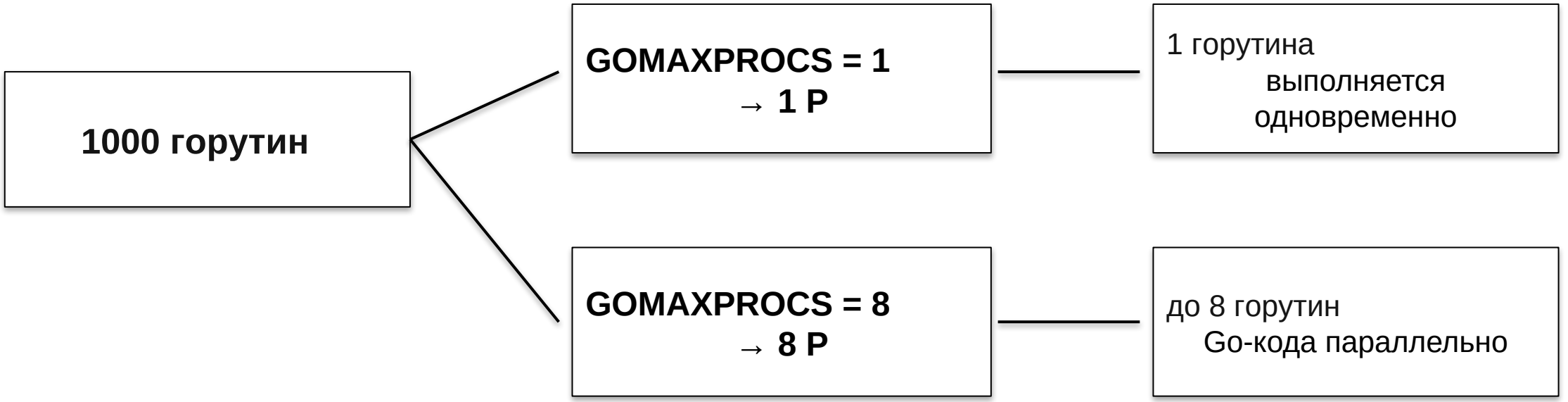
Модели G → M недостаточно: рантайму нужны ресурсы исполнения, которые нельзя навсегда закрепить за одним потоком ОС.

P содержит локальную очередь готовых G и другое собственное состояние.

M может блокироваться в системном вызове и временно отдавать P.

Освободившийся P подхватит другой M и продолжит выполнение Go-кода.

Конкурентность ≠ параллелизм



GOMAXPROCS

Практическая модель: GOMAXPROCS ≈ число P ≈ предел параллелизма Go-кода.

Это не число горутин и не максимальное количество потоков ОС.

Нагрузка на CPU ограничена доступным числом P и вычислительных ресурсов.

При большом числе ожидающих горутин одновременно исполняется лишь небольшая их часть.

GOMAXPROCS в контейнере

На хосте может быть 128 CPU, а лимит cgroup в контейнере — всего 2 CPU.

Избыточный параллелизм исчерпывает квоту CPU → ограничение ядром → рост хвостовых задержек.

Современный Go учитывает ограничения CPU контейнера при выборе GOMAXPROCS.

Цепочка: GOMAXPROCS → P → M → планировщик Linux → cgroups → p99.

Где ждут готовые к выполнению горутины



Локальные очереди уменьшают конкуренцию. Общая нужна для координации и передачи работы между P.

Что делает `go f()`

Компилятор генерирует вызов рантайма для создания горутины.

Рантайм создает G и помещает ее в очередь готовых текущего P.

Локальная очередь P в текущей версии имеет 256 ячеек и поле runnext.

Создание горутины не означает немедленного выполнения функции.

```
      go worker()
    ↓
новая G
    ↓
runqput(P, G)
    ↓
планировщик
```

Кража работы: когда одному P нечего исполнять



Почему воровать пачку работы

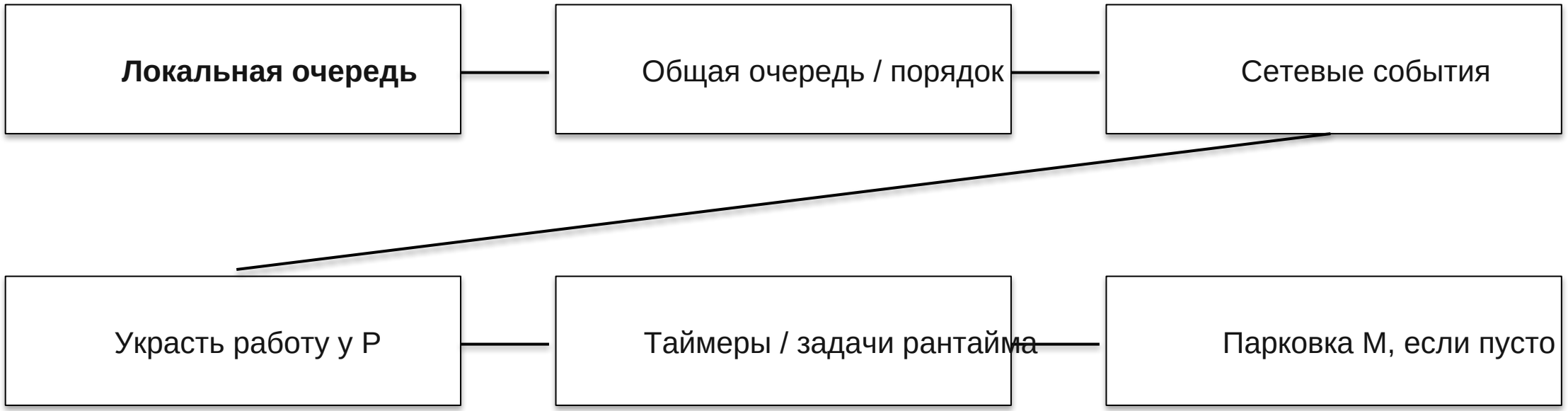
Кража одной G: быстро закончили → снова поиск → дополнительные расходы.

Кража всех G: исходный P сам остается без работы.

Обычный компромисс — забрать примерно половину готовой работы.

Балансируем нагрузку без центрального планировщика и общего лока на каждую операцию.

Упрощенный цикл планировщика



Блокировка G ≠ блокировка потока ОС

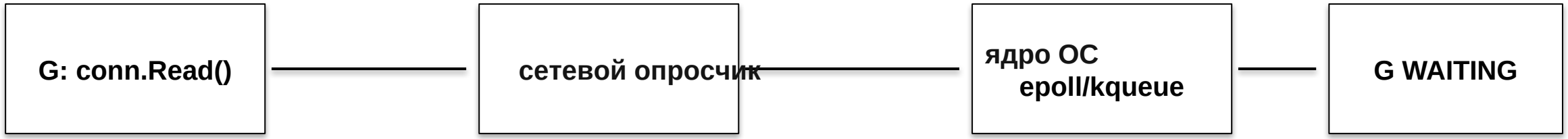
Ожидание канала, mutex, таймера и других примитивов переводит G в WAITING.

M в это время может выполнять другую готовую горутину.

Планировщик сохраняет состояние G и возвращает ее в очередь готовых после наступления события.

Поэтому тысячи ожидающих горутин не требуют тысячи потоков ОС.

Сетевой опросчик: ожидание I/O без потока на соединение



данные готовы → G переходит в RUNNABLE → планировщик запускает ее снова

Код выглядит синхронным, но ожидание сетевого сокета управляется планировщиком.

Блокирующий системный вызов: M может ждать в ядре



Вернувшийся M1 должен получить P; если свободных P нет, G1 продолжит работу позднее через планировщик.

Сетевой опросчик и блокирующий вызов — разные механизмы

Ожидание сетевого I/O	Блокирующий системный вызов
G ждет M свободен для другой G Поток ОС не обязан ждать	G + M действительно могут ждать в ядре P отделяется другой M продолжает Go-код

Всегда спрашиваем: ЧЕГО ждет горутина? Именно это определяет механику рантайма.

Вытеснение: что если G не блокируется сама

Вычислительный цикл не должен бесконечно занимать P.

Начиная с Go 1.14 рантайм поддерживает асинхронное вытеснение.

Текущий внутренний ориентир — около 10 мс, но это не гарантированный квант времени.

Планирование нестрогое: влияют также ядро ОС, GC и нагрузка.

Стек горутины: небольшой и растущий



Нет места → выделить больший стек → скопировать кадры → исправить ссылки

Рантайм может подбирать начальный размер стека выше минимума
по статистике нагрузки.

Почему рост стека — сложнее обычного memcpy

После переноса адрес стека меняется.

Указатели внутрь прежнего стека необходимо скорректировать.

Карты стека от компилятора описывают расположение указателей в кадрах.

Служебные структуры defer, panic и ожидания каналов тоже требуют исправления.

У M есть служебный стек — g0

Горутина выполняется на собственном растущем стеке.

Планировщик и часть рантайма работают на служебном стеке M (g0).

Это позволяет исполнять код, который сам не должен зависеть от роста и вытеснения обычной горутины.

M переключается между пользовательской G и своей служебной g0.

Сколько стоит горутина

Горутина значительно дешевле потока ОС при массовом ожидании,
но тоже расходует ресурсы.

Цена: метаданные G + стек + работа планировщика + аллокации + сканирование GC.

100 000 × 2 КиБ ≈ 195 МиБ — лишь грубая оценка памяти стеков снизу.

Реальное потребление может быть выше из-за объектов в хипе, буферов,
таймеров, сокетов и выросших стеков.

Утечка горутин: планировщик исправен, архитектура — нет

```
func process() {
    ch := make(chan Result)

    go func() {
        ch <- slowOperation()
    }()

    return // никто не читает ch
}
```

1 запрос → +1 зависшая G
1000 запросов → +1000 G

Симптомы:
число горутин ↑
память ↑
GC CPU ↑
задержка ↑

Планировщик не ограничивает поступление работы

Миллион горутин перед PostgreSQL с пулом из 20 соединений — плохая идея.

Планировщик распределяет готовые G, но не знает возможности БД или внешнего API.

Нужны лимиты параллелизма, семафор, пул работников, ограниченные очереди.

Дешевая горутина позволяет незаметно создать огромную очередь прямо в памяти процесса.

Диагностика рантайма и планировщика

`runtime.NumGoroutine()` и метрики — динамика числа G.

Профиль горутин pprof — где и чего ждут горутины.

`go tool trace` — планирование G, сетевые ожидания, системные вызовы, GC.

`GODEBUG=schedtrace=1000,scheddetail=1` — снимок состояния планировщика.

Что ограничивает: CPU, сеть, системные вызовы, локи, пул соединений или утечка?

Сравнение Java-потоков и горутин Go

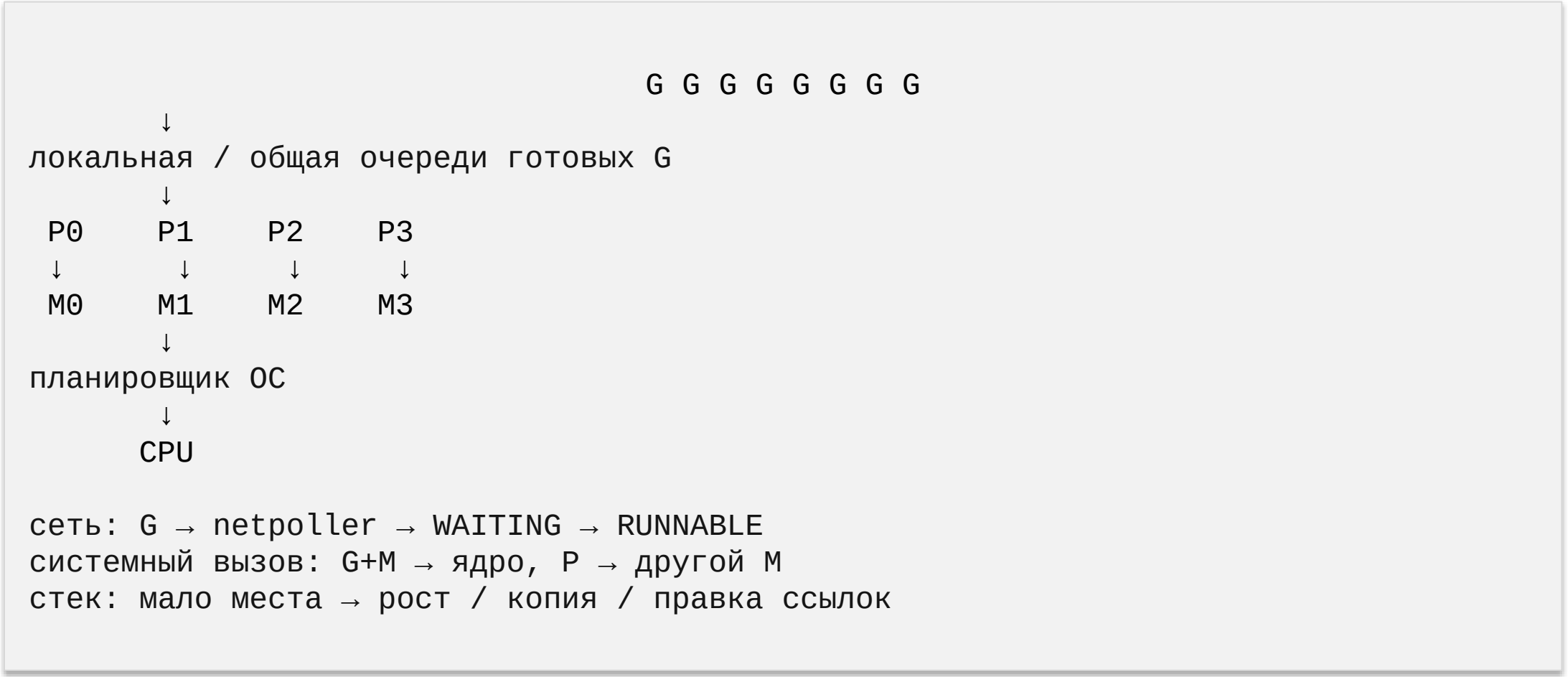
Поток Java
≈ поток ОС

Виртуальный поток Java
пользовательская задача → рабочий поток ОС

Горутина Go
G → планировщик Go → M

Дешевый контекст выполнения не делает CPU, соединения к БД и возможности внешних сервисов бесконечными.

Финальная модель



Что важно запомнить

G ≠ поток ОС. G — контекст выполнения; M — поток ОС; P — ресурс Go-кода.

GOMAXPROCS задает предел параллелизма через P, а не число G или M.

Локальные очереди и кража работы уменьшают конкуренцию и выравнивают нагрузку.

Сетевой опросчик позволяет тысячам горутин ждать I/O без потока на соединение.

Блокирующий системный вызов удерживает M; P может перейти другому M.

Растущие стеки уменьшают стартовую цену G, но горутина все равно
использует память, GC и планировщик.

Планировщик не ограничивает нагрузку на зависимости и не управляет жизненным циклом.

Планировщик Go: глубокий разбор

Путь одной горутины через runtime/proc.go

```
newproc → runqput → schedule → findRunnable → execute
→ gopark / goready → runqsteal
```

Главный вопрос

Что происходит после go worker()?

- Кто создает G и где она оказывается?
- Как планировщик выбирает следующую горутину?
- Что происходит при блокировке и пробуждении?
- Как P забирает работу у другого P?

Полный путь создания новой горутины

```
go worker()
↓
newproc → newproc1 → _Grunnable → runqput
↓
schedule → findRunnable → execute
↓
_Grunning → worker()
```

Создание и выполнение — разные этапы: сначала G становится RUNNABLE.

go f() не означает немедленный запуск

```
func main() {
go worker()
}
```

Компилятор вызывает runtime.newproc.
Рантайм создает или переиспользует G, помечает ее RUNNABLE и ставит в очередь.
Когда она получит CPU — решает планировщик.

Почему рантайму нужен служебный стек

стек пользовательской G

↓ systemstack

служебный стек M.g0

↓

создание G / метаданные / операции очередей

Низкоуровневые операции планировщика выполняются на стеке M.g0.

newproc1: G можно переиспользовать

```
go worker()
```

↓

gfget(P): есть свободная G?

да → переиспользовать G

нет → выделить G и стек

↓

подготовить контекст выполнения

↓

```
status = _Grunnable
```

Завершившиеся G рантайм сохраняет в пуле и переиспользует.

P: runnext и локальная очередь готовых G

```
P0
├─ runnext → G42
└─ runq[256] → G10 G11 G12 ...
```

runnext — кандидат на быстрый следующий запуск.

runq — локальная кольцевая очередь готовых горутин.

Локальная очередь уменьшает конкуренцию за общий ресурс.

Зачем существует runnext

Родительская G создает дочернюю:

```
go child()
```

Если отправить ее в хвост длинной очереди, возрастет задержка планирования.
runnext ускоряет передачу управления между связанными горутинами.
Это оптимизация задержки, без строгого приоритета.

Переполнение локальной очереди

Локальная очередь P0: около 256 G

\+ новая G

↓

runqputslow

↓

примерно половина остается локально

другая часть уходит пакетом в общую очередь

Перегруженный P освобождает место и делится работой с другими P.

wakep: почему нельзя будить поток на каждую G

Парковка и пробуждение потока ОС имеют цену.

Если будить M на каждую новую G:
работа появилась → поток пробужден → другой M уже все забрал → снова сон.

Рантайм будит M осторожно: нужен свободный P и недостаточно
уже активно ищущих работу M.

M в режиме активного поиска

M ИЩЕТ РАБОТУ

↓

активно проверяет источники:

локальная / общая очередь / сеть / кража

↓

не нашел

↓

ПАРКОВКА

Сразу парковаться — дорогие повторные пробуждения.
Крутиться вечно — зря тратить CPU.
Активный поиск — компромисс между задержкой и расходом CPU.

schedule(): один цикл планировщика

```
schedule()
↓
findRunnable()
↓
execute(gp, inheritTime)
```

Задача schedule: найти готовую G и передать ей исполнение.
Сложность находится в поиске работы и безопасной смене состояний.

findRunnable(): работа хранится в разных местах

findRunnable() проверяет:

1. задачи рантайма / GC / таймеры
2. периодически — общую очередь ради справедливости
3. локальную очередь
4. общую очередь
5. сетевые события без блокировки
6. кражу работы
7. сетевое ожидание / парковку

Точный порядок — деталь реализации рантайма.

Локальность и справедливость

Локальная очередь:
+ меньше общей координации
+ лучше локальность кешей

Но постоянный выбор локальной работы может обделять общую очередь.
Поэтому рантайм периодически проверяет общую очередь первой.

Планировщик балансирует локальность и справедливость.

Общая очередь: забираем работу пакетом

ОБЩАЯ ОЧЕРЕДЬ
G1 G2 G3 G4 G5 ...

забрать пакет

P2 получает:

- одну G для немедленного выполнения
- остальные — в локальную очередь

Один доступ к общей очереди окупается серией локальных запусков.

Сетевой опросчик перед дорогой кражей работы

Локальная и общая очереди пусты.

В ядре могли появиться готовые к чтению/записи сокеты.
Неблокирующий сетевой опрос возвращает готовые G.

Получить готовую сетевую G нередко дешевле, чем обходить P
и искать чужие очереди.

Кража работы

```
До:
P0: G1 G2 G3 G4 G5 G6 G7 G8
P1: —
```

P1 забирает работу у P0

```
После примерно:
P0: G1 G2 G3 G4
```

P1: G5 G6 G7 + одну G исполняет сразу

Пустой P не должен простаивать, пока у другого есть готовая работа.

Почему крадём примерно половину

Забрать одну G:
→ скоро снова искать работу
→ дополнительные расходы планировщика

Забрать все:
→ у исходного P ничего не останется

Половина — компромисс: один P получает запас, второй не простаивает.

execute(): G получает M

```
findRunnable вернул gp
↓
execute(gp)
↓
M.curg = G
G.m = M
_Grunnable → _Grunning
↓
gogo(&G.sched)
↓
```

восстановить PC / SP / регистры

↓

продолжить Go-код

Планировщик не исполняет код по инструкциям

После gogo CPU непосредственно выполняет пользовательский код.

Планировщик возвращается, когда происходит событие:

- блокировка
- системный вызов
- вытеснение
- Gosched
- рост стека
- panic / завершение

Планировщик реагирует на события, он не интерпретатор.

gopark: горутина пока не может работать

```
G42 RUNNING
↓
gopark
↓
```
перейти на g0 / park_m
```
↓
_Grunning → _Gwaiting
↓
```
отвязать G от M
```
↓
```
выбрать другую G

Блокируется G, а не поток ОС — для примитивов, известных рантайму.

goready: событие произошло

событие наступило

↓

`goready(G42)`

↓

`ready()`

↓

`_Gwaiting → _Grunnable`

↓

`runqput`

↓

wakep при необходимости

Важно: goready не запускает G сразу — она лишь становится RUNNABLE.

Ожидание канала: полный жизненный цикл

`G1 RUNNING`

получение из пустого канала

зарегистрировать ожидающую G

↓

`gopark → G1 WAITING`

M исполняет другую G

↓

`G2` другая G отправляет значение

↓

`goready(G1) → RUNNABLE`

↓

`scheduler → execute`

G снова RUNNING, получение завершено

Почему парковка и регистрация ожидания должны быть согласованы

Опасная гонка:

G1: проверила → пусто
[переключение]
G2: отправила → получателя нет → завершила
G1: припарковалась навсегда

Это потерянное пробуждение.

Примитив синхронизации должен связать регистрацию ожидания с парковкой так, чтобы сигнал не терялся.

Gosched / вытеснение ≠ парковка

gopark:
RUNNING → WAITING
G сейчас НЕ МОЖЕТ продолжать

Gosched / вытеснение:
RUNNING → RUNNABLE
G МОЖЕТ работать, но уступает ресурс исполнения

Причины переключения планировщика различаются.

Завершение горутины

worker() завершилась
↓
goexit / gdestroy
↓
_Grunning → _Gdead
↓
G может вернуться в пул
↓
позже — повторное использование через gfget

Объект G может жить дольше одного запуска горутины.

Полный жизненный цикл одной G

```
go f() → newproc → newproc1
↓
RUNNABLE → runqput
↓
schedule → findRunnable → execute
↓
RUNNING
```

+ блокировка

```
gopark → WAITING
```

+ событие

```
goready → RUNNABLE
↓
execute → RUNNING
```

+ завершение

```
goexit → DEAD → pool
```

Что происходит под нагрузкой backend

100 000 HTTP-горутин ≠ 100 000 одновременно исполняемых задач.

Например:
20 RUNNING
500 RUNNABLE
70 000 ждут сеть
20 000 ждут пул БД
9 480 ждут таймер/канал/mutex

Диагностика начинается с состояний и причин ожидания, а не числа G.

Задержка планирования

G стала RUNNABLE в момент t0
execute начался через 200 мс

G была готова, но не получила ресурс исполнения.

Возможные причины:

- CPU перегружен
- слишком длинные очереди готовых G
- GOMAXPROCS / ограничение cgroup
- работа GC и рантайма

Не путать с WAITING при ожидании сети или БД.

Диагностика: go tool trace

Трассировка показывает:

G создана → готова → исполняется → ждет → снова готова

Смотрим:

- задержку планирования
- причины блокировок
- ожидание сети
- системные вызовы
- GC
- активность P/M

Обработчик медленный из-за вычислений или ожидания CPU?

Что важно запомнить

- go f() создает готовую G, не новый поток ОС
- P хранит локальную очередь и runnext
- schedule = findRunnable → execute
- локальные очереди дают локальность, общая — координацию
- кража работы выравнивает нагрузку P
- gopark: RUNNING → WAITING
- goready: WAITING → RUNNABLE
- планировщик исполняет G, но не ограничивает входящую нагрузку
- под нагрузкой важны RUNNING / RUNNABLE / WAITING и причина ожидания

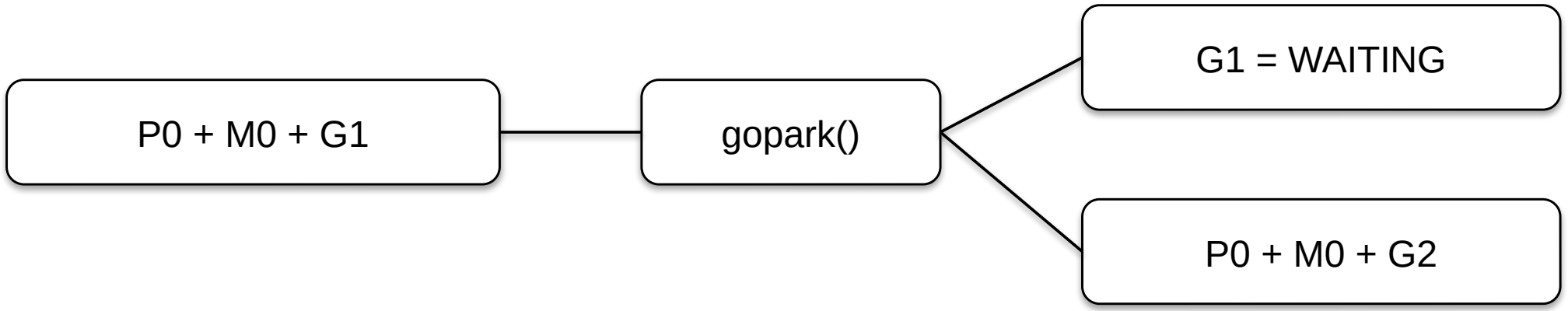
Рантайм Go: внутренние механизмы синхронизации

gopark / goready / sudog / семафор / sync.Mutex

Главный вопрос

- Что происходит, когда `<-ch` или `mu.Lock()` пока не могут продолжить выполнение?
- Почему ожидающая горутина обычно не удерживает поток ОС?
- Как рантайм запоминает ожидающую G и возвращает ее планировщику?

Блокируем G, а не M

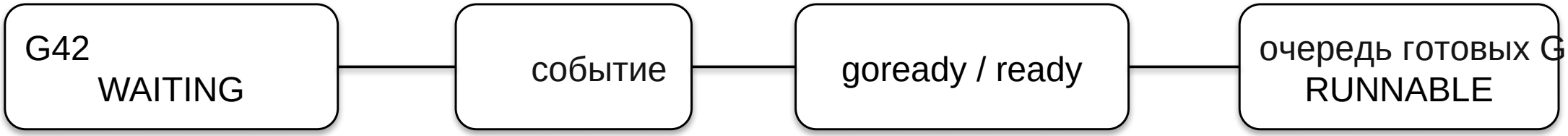


Ожидание, известное планировщику, паркует G: M и P могут исполнять другие готовые горутины.

gopark: RUNNING → WAITING

- `gopark`: текущая горутина не может продолжать выполнение.
- Рантайм переходит на служебный стек g0, помечает G как `_Gwaiting` и отвязывает от M.
- M возвращается в `schedule()` и получает другую готовую G.

goready: WAITING → RUNNABLE

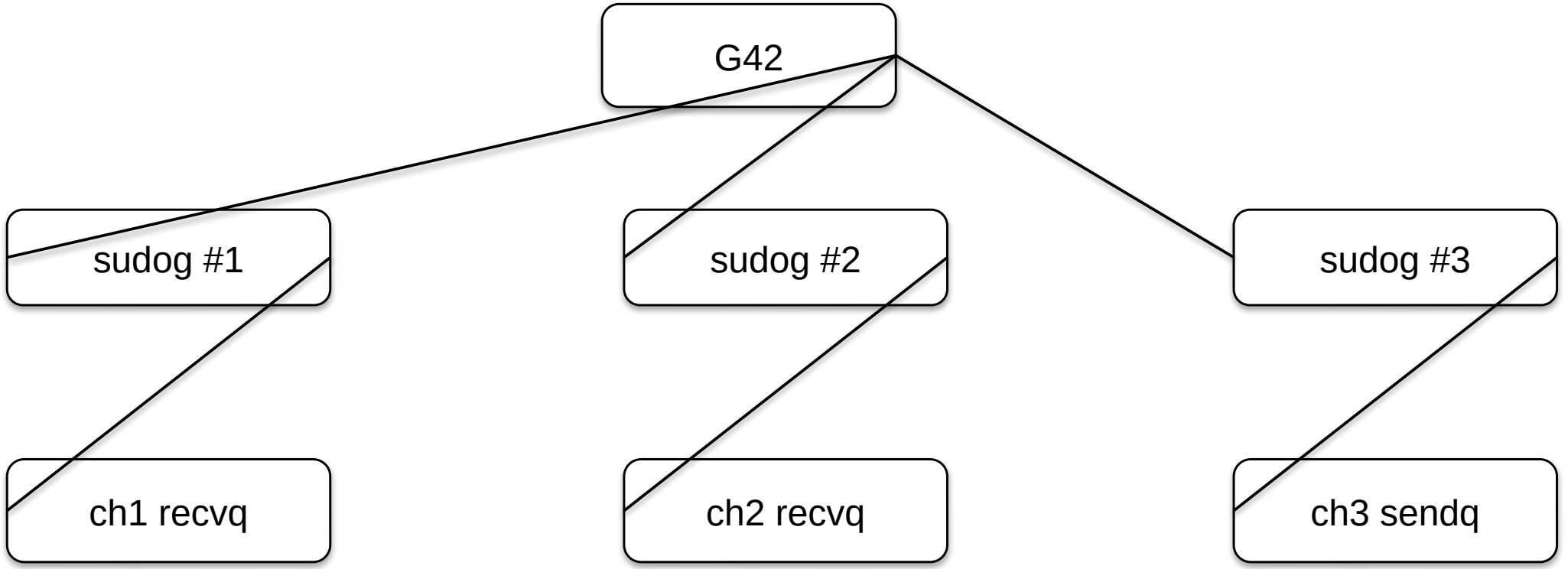


`goready` не запускает G немедленно — она снова претендует на P/M.

Что такое sudog

- `sudog` — запись горутины в очереди ожидания конкретного объекта синхронизации.
- Она связывает G с каналом, семафором и другими структурами ожидания.
- `sudog` — отдельный узел очереди, не сама горутина.

Почему не хранить просто *g



`select` регистрирует одну G сразу в нескольких очередях ожидания:
для этого нужна связь через несколько `sudog`.

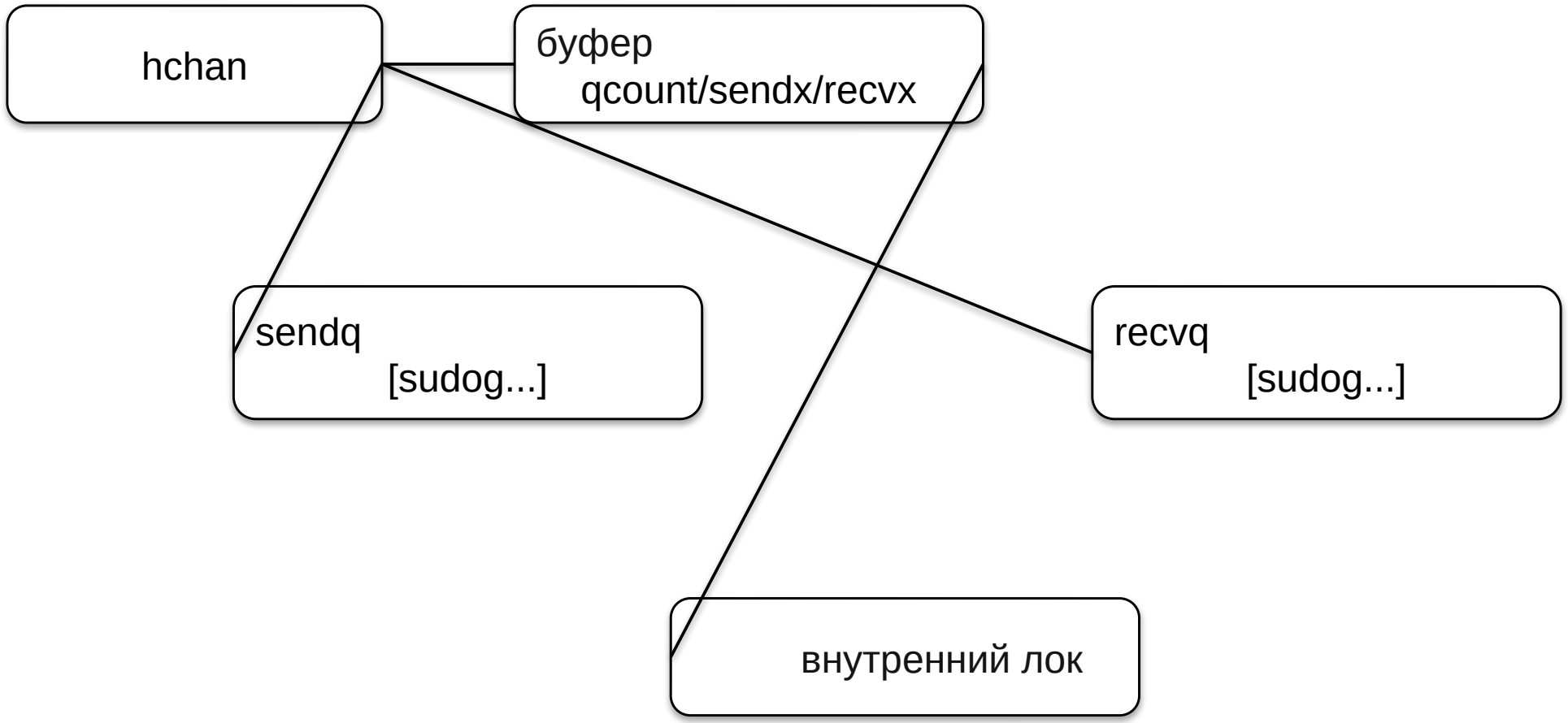
Пул sudog

• Ожидания часты: выделять новый `sudog` при каждой блокировке было бы дорого.

• Рантайм переиспользует узлы через `acquireSudog()` / `releaseSudog()`.

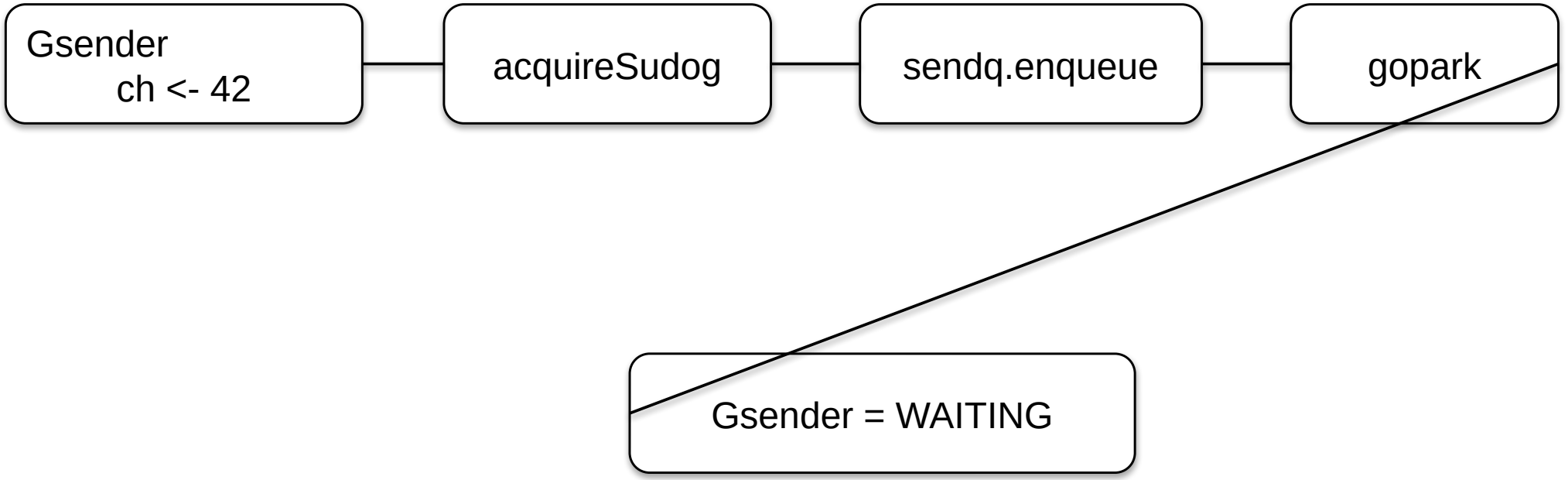
• Типичный подход: дешевый быстрый путь и пул для часто используемых служебных объектов.

Канал: внутренние очереди ожидания



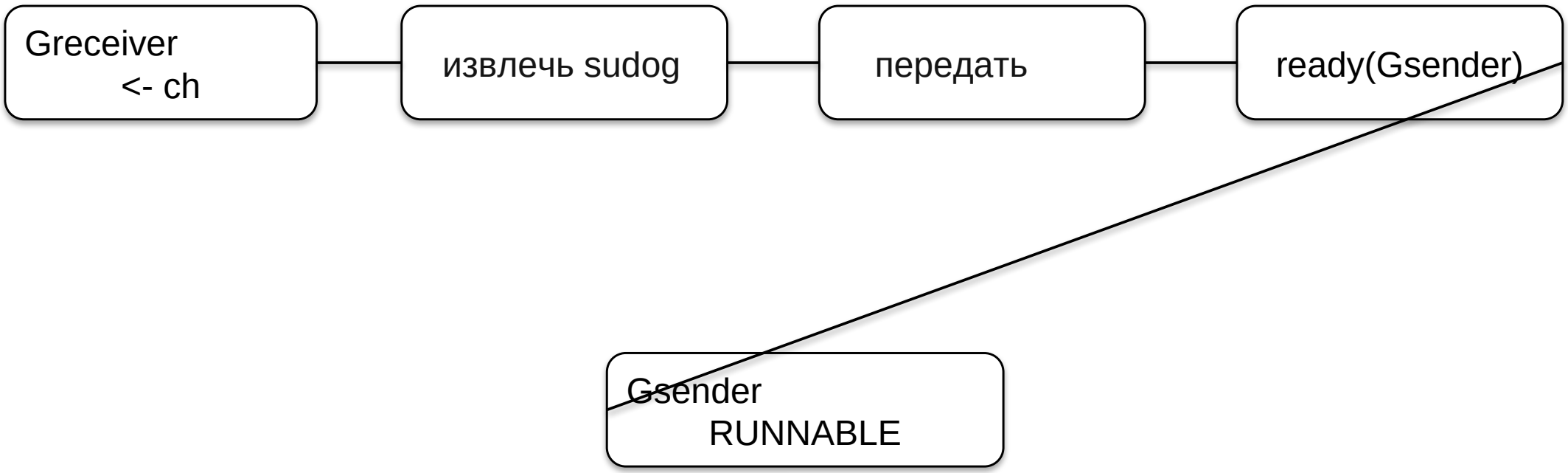
`sendq` хранит отправителей, `recvq` — получателей. Лок защищает состояние канала, но не все время ожидания горутины.

Отправка без буфера: получатель отсутствует



`sudog` хранит G и адрес значения до встречи с получателем.

Получатель приходит позже



Отправитель снова проходит через планировщик; получатель не передает ему CPU.

Потерянное пробуждение: главная гонка ожидания

• Неправильно: проверить условие → переключиться → отправить сигнал до регистрации ожидания → припарковаться.

• Чтобы не потерять сигнал, регистрация, освобождение лока и парковка должны быть согласованы.

Зачем `gopark` получает функцию освобождения лока

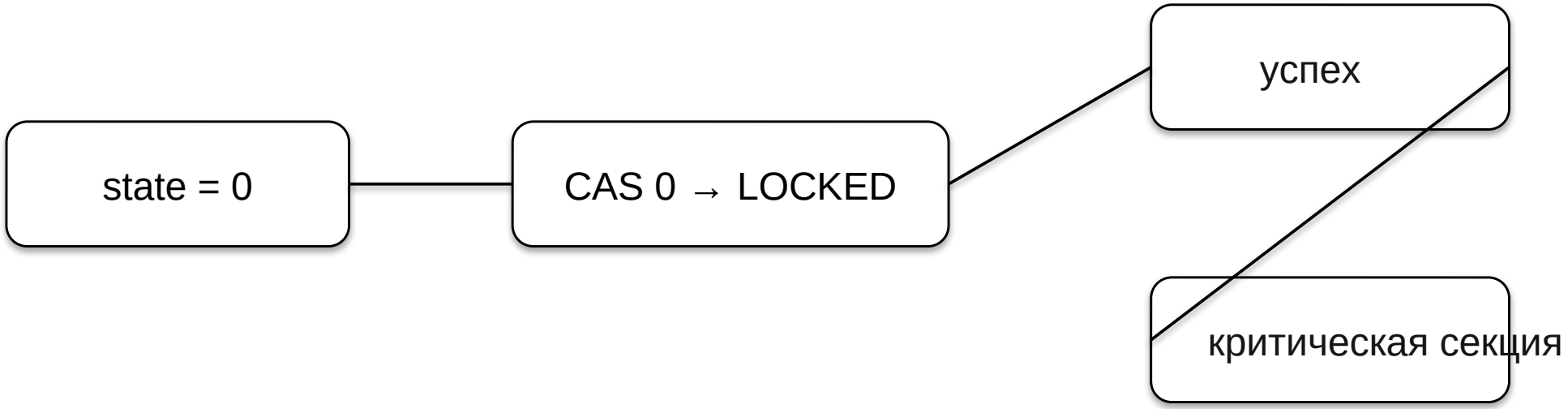
- Рантайм освобождает внутренний лок после регистрации ожидающего, но до его сна.
- Сигнализирующая сторона видит G либо под локом, либо уже в очереди ожидания.
- Это предотвращает потерю пробуждения; сходная проблема есть у условных переменных, futex и семафоров.

sync.Mutex: небольшая структура, сложный протокол

```
type Mutex struct {
    state int32
    sema  uint32
}

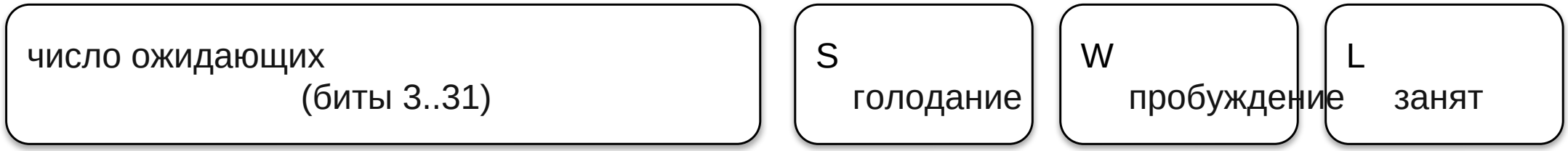
        // state: занят / пробужден / голодание / число ожидающих
```

Быстрый путь Mutex: CAS



Без конкуренции: CAS и возврат. Нет `sudog`, семафора или парковки.

Состояние Mutex: упакованные биты

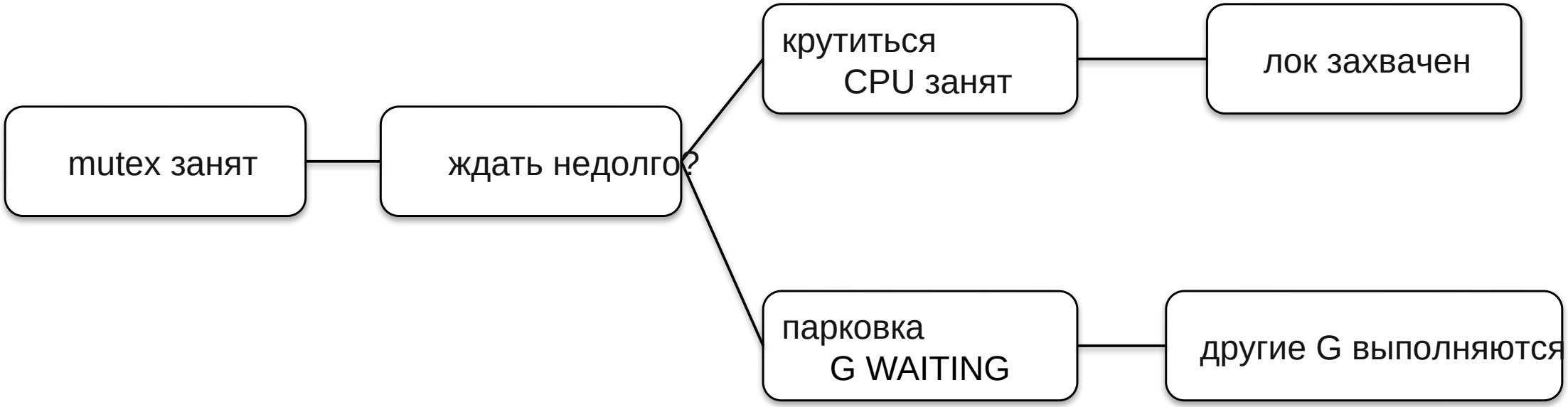


Один `int32` хранит флаги, режим голодания и число ожидающих.

Медленный путь захвата: зачем активное ожидание

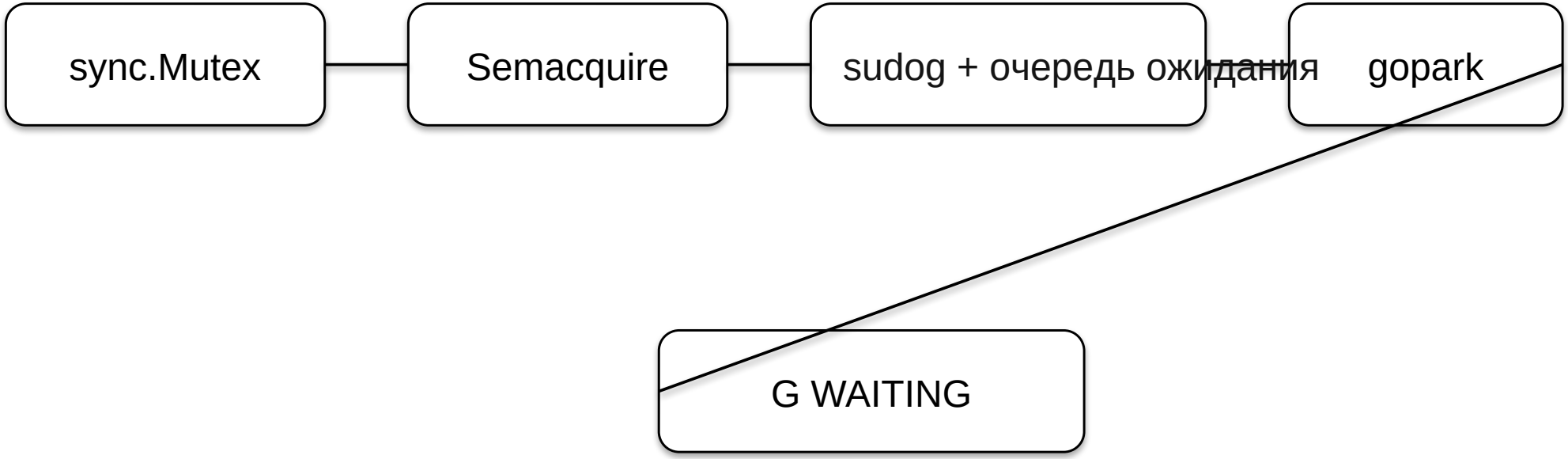
- CAS проигран, но критическая секция может завершиться быстрее парковки и пробуждения.
- Рантайм допускает короткое активное ожидание: G остается RUNNING и еще несколько раз проверяет лок.
- Цена: CPU в обмен на меньшую задержку синхронизации.

Активное ожидание или парковка



Рантайм осторожно использует активное ожидание, учитывая число CPU и наличие более полезной работы на текущем P.

Семафор рантайма: мост к планировщику

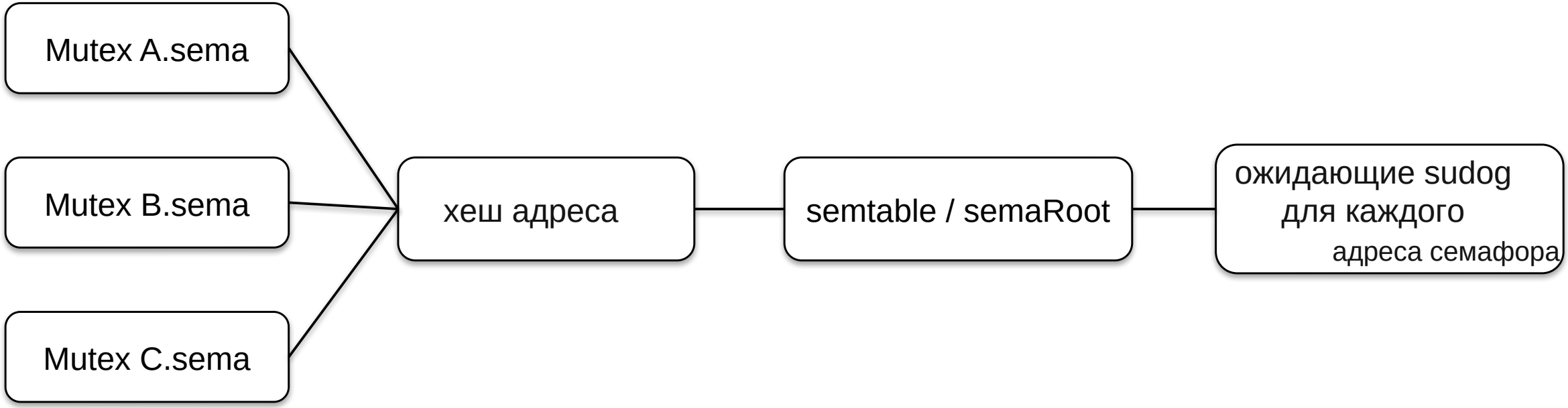


Это семафор рантайма: он паркует/пробуждает горутины,

`semacquire1`: зачем повторять проверку условия

- Быстрый путь атомарно пытается получить семафор.
- Если ресурса нет: увеличиваем число ожидающих, затем проверяем его еще раз перед постановкой в очередь и парковкой.
- Повторная проверка закрывает гонку между первым чтением условия и регистрацией ожидания.

Где хранятся ожидающие Mutex



Очередь не лежит в самом Mutex: тяжелые структуры общие и нужны только при конкуренции.

Освобождение → Semrelease → пробуждение

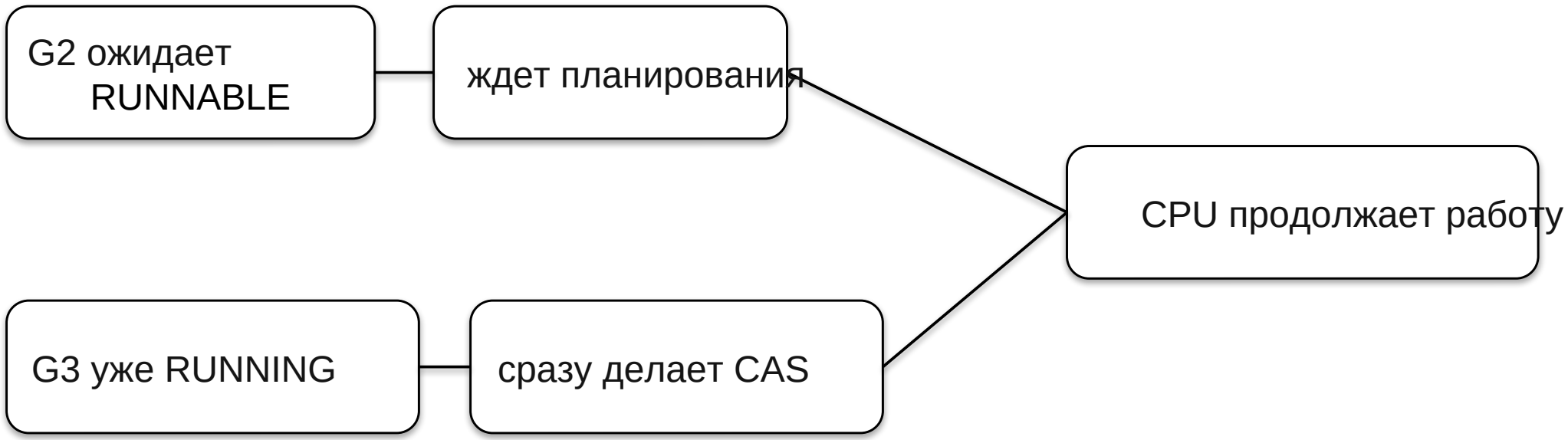


Быстрое освобождение — одна атомарная операция; планировщик нужен только при ожидающих горутинах.

Обычный режим: перехват лока новой G

- Пробужденная G должна получить CPU и снова попытаться захватить mutex.
- Уже исполняющаяся новая горутина может обогнать прежнюю ожидающую G.
- Это повышает пропускную способность, но увеличивает риск долгого ожидания отдельных горутин.

Почему перехват лока повышает пропускную способность



Обычный режим предпочитает уже исполняющуюся G: меньше переключений

и выше про

Режим голодания

• Если G слишком долго проигрывает, mutex может перейти в режим голодания.

• Владение передается следующей ожидающей G; новые участники не должны перехватывать лок.

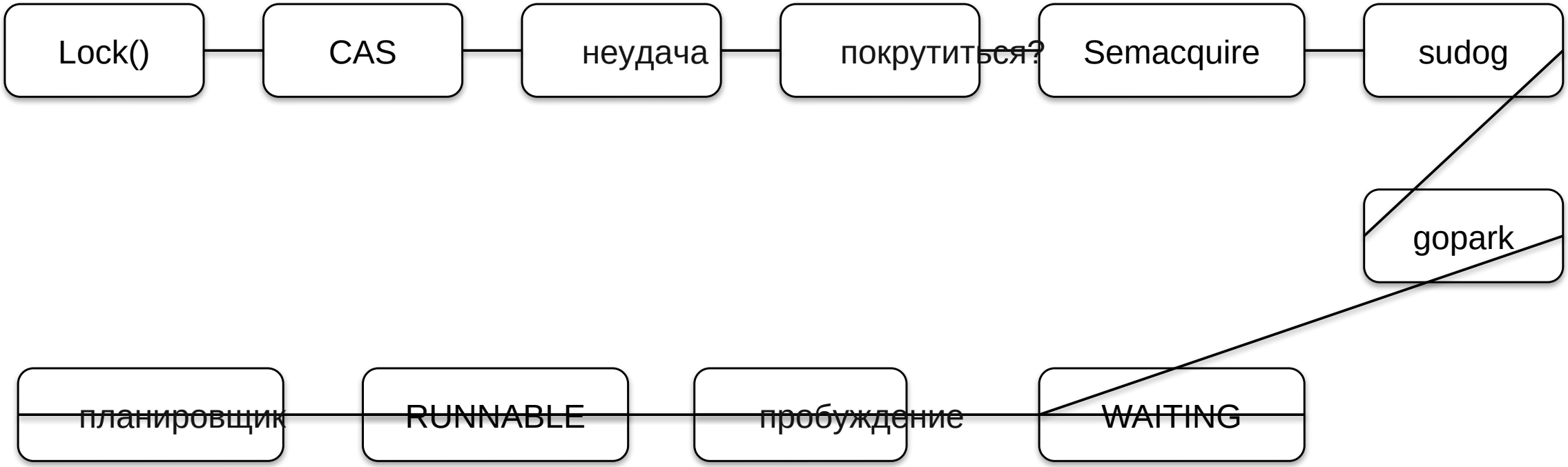
• Компромисс: справедливость и p99 лучше, пропускная способность ниже. Порог около 1 мс — деталь реализации, не гарантия API.

Обычный режим и режим голодания

ОБЫЧНЫЙ	ГОЛОДАНИЕ
Освобождение	Освобождение
разбудить G	прямая передача
новая G может перехватить лок	новые G ждут

Обычный режим ускоряет работу; голодание повышает справедливость.

Полный медленный путь Mutex.Lock



Быстрый путь заканчивается на CAS. Остальная механика возникает

только при

Канал и Mutex: единая схема ожидания

- В канале очереди `sendq/recvq` находятся внутри `hchan`.
- Для Mutex ожидающих обслуживает общая таблица семафоров рантайма.
- В обоих случаях: регистрация → `sudog` → `gopark` → событие → `goready` → RUNNABLE.

Что происходит при конкуренции

- Много горутин могут ждать один mutex при невысокой загрузке CPU.
- Узкое место — последовательный доступ, а не нехватка планировщика.
- Конкуренция добавляет активное ожидание, парковки, пробуждения и задержку планирования вокруг критической секции.

Backend-пример: глобальный mutex кеша

- При 10 RPS: Lock → map lookup → Unlock выглядит бесплатным.
- При 50 000 RPS: тысячи G конкурируют за критическую секцию.
- Растет p99; профили блокировок и mutex указывают горячую точку, хотя CPU может быть далеко от 100%.

Диагностика

- Профиль mutex: где теряется время на захват конкурентных локов.
- Профиль блокировок: ожидания каналов, Mutex, Cond и `select`.
- `go tool trace`: RUNNING → WAITING → RUNNABLE → RUNNING.
- `runtime.NumGoroutine` недостаточно: нужны причина и время ожидания.

Практика

- Измерить Mutex без конкуренции: доказать дешевизну CAS.
- `RunParallel` и разные длительности критической секции: увидеть активное ожидание и парковку.
- Канал без буфера: в трассировке увидеть `sendq/recvq` и пробуждение.

Что важно запомнить

- `gopark` паркует G; `goready` возвращает ее в RUNNABLE.
- `sudog` — запись G в очереди ожидания конкретного объекта.
- Быстрый Mutex — атомики; медленный — ожидание и семафор.
- Очереди семафоров паркуют G без выделенного ожидающего потока ОС.

- Обычный режим и голодание балансируют скорость и справедливость.
- Под нагрузкой важна очередь перед критической секцией,
а не сама инструкция захвата лока.

sync.Mutex.state и lockSlow()

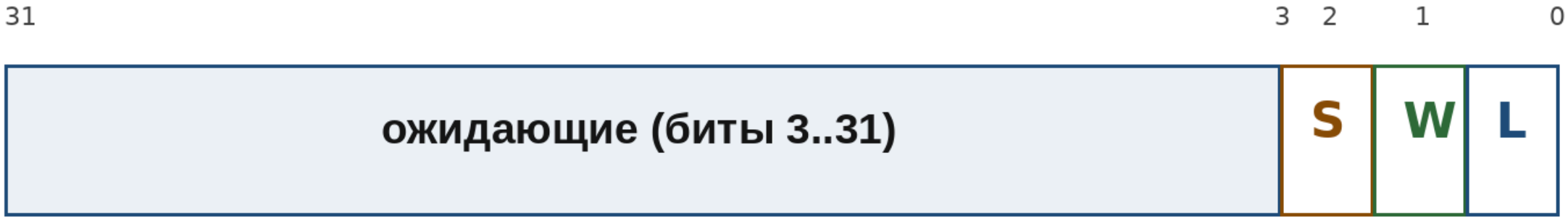
- Цель: разобраться в адаптивном протоколе Mutex, а не только в Lock/Unlock.
- Путь: биты состояния → CAS → активное ожидание → учет G → семафор → передача.
- Точные биты и пороги — детали текущей реализации рантайма Go.

Базовая структура Mutex

```
type Mutex struct {
    state int32
    sema  uint32
}
```

Вся логика конкуренции упакована в 32-битный state и семафор рантайма.

Mutex.state: 32 бита одного автомата состояний



L	блокировка захвачена
W	одна G уже назначена для пробуждения
S	режим голодания: очередь получает приоритет
waiters	число ожидающих = state >> 3

Константы state

```
const (
    mutexLocked = 1 << iota
    mutexWoken
    mutexStarving
    mutexWaiterShift = iota
)
```

После трех флагов старшие биты хранят число ожидающих G.

Как читать state

Примеры

- `state = 0` → свободен
- `state = 1` → Locked
- `state = 9` → Locked + 1 waiter
- `state = 11` → Locked + Woken + 1 waiter

Формула

- `waiters = state >> 3`
- `+8` означает ещё одного waiter
- младшие три бита — режим/координация

Зачем все в одном int32?

- Связанные признаки меняются одним CAS: прежнее → новое состояние.
- Отдельный лок для самого Mutex.state не нужен.
- Автомат состояний: прочитать → вычислить → CAS, при конфликте повторить попытку.

Lock(): обычный путь должен быть почти бесплатным



Без конкуренции не нужны планировщик, семафор и sudog.

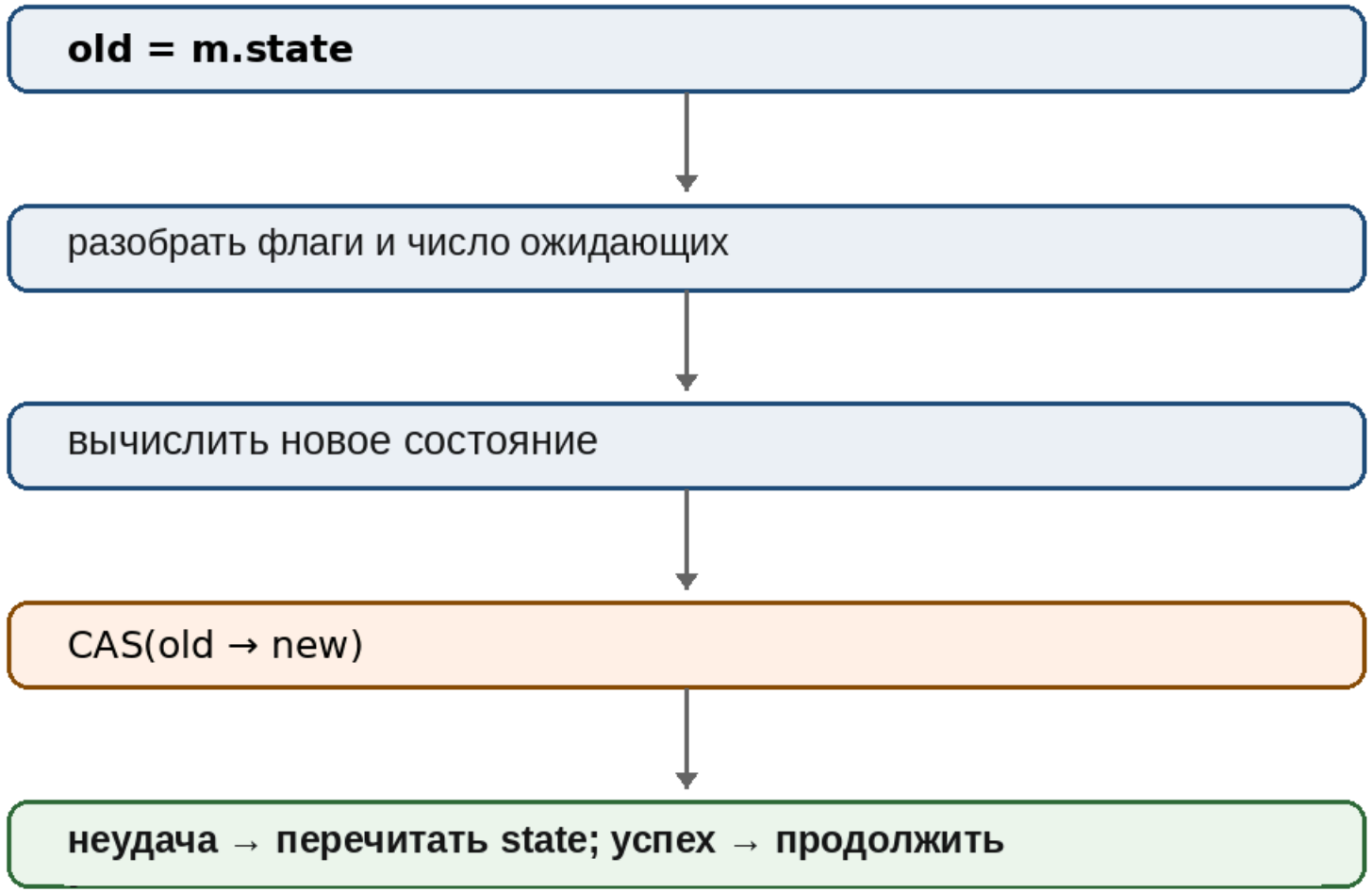
Быстрый путь

```
if atomic.CompareAndSwapInt32(
    &m.state,
    0,
    mutexLocked,
) {
    return
}

m.lockSlow()
```

Вся сложность вынесена из частого пути в lockSlow().

lockSlow(): автомат состояний на CAS



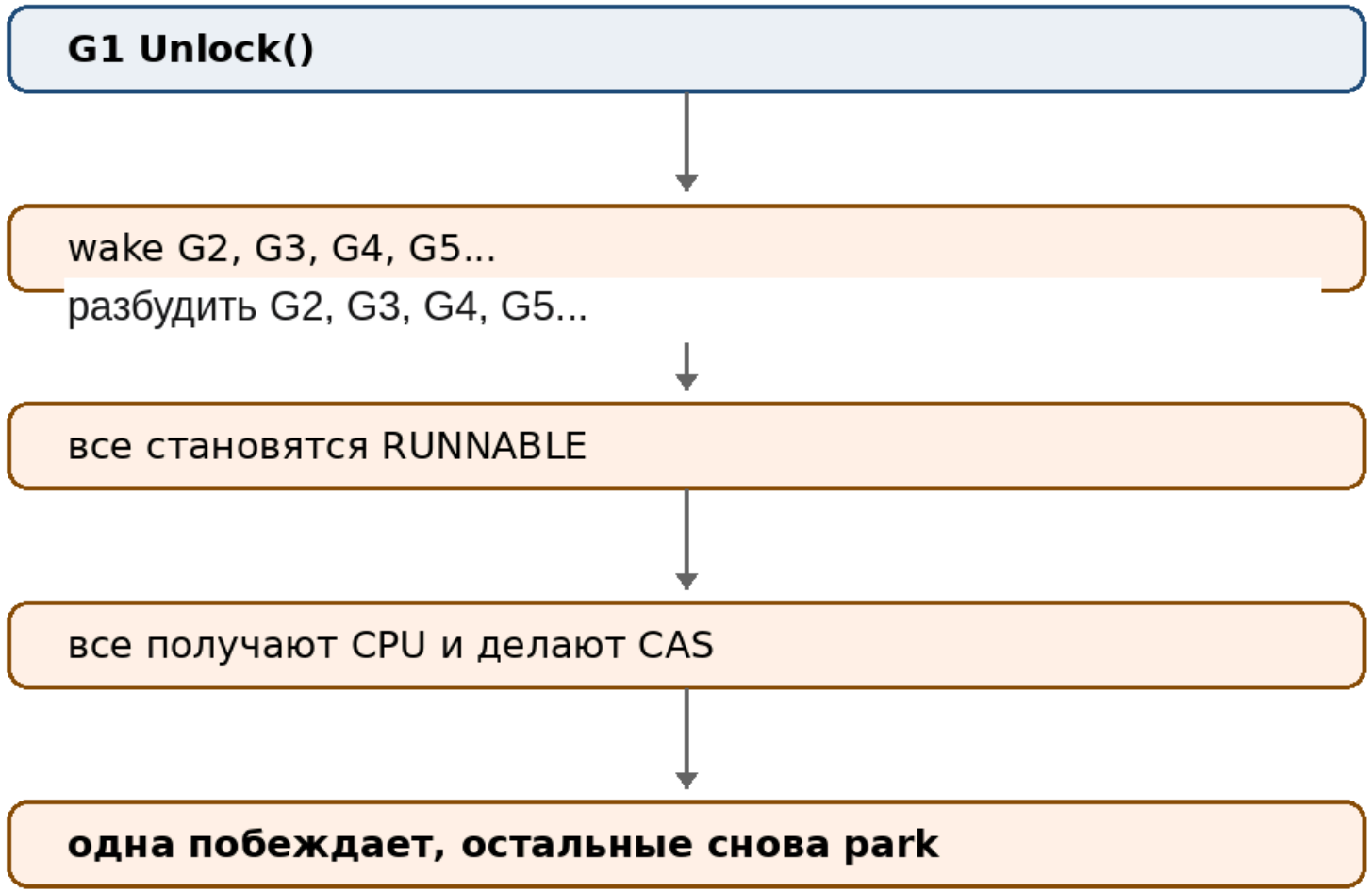
Локальные переменные lockSlow()

- waitStartTime — начало реального ожидания.
- starving — эта G ждет слишком долго.
- awoke — попытка захвата уже назначена на пробуждение.
- iter — число итераций активного ожидания.

mutexWoken: зачем отдельный бит?

- Mutex может иметь спящие G и уже пробужденного претендента.
- mutexWoken=1 сообщает Unlock: еще одну G будить не нужно.
- Меньше циклов: парковка → пробуждение → проигрыш CAS → парковка.

Без mutexWoken: массовое пробуждение



Одно освобождение лока не требует будить целую толпу.

awoke и mutexWoken

awoke

- локальная переменная одного Lock()
- значит: именно эта G уже активный кандидат
- позже G должна сбросить флаг пробуждения

mutexWoken

- глобальный бит в m.state
- согласует Unlock и конкурентов
- предотвращает лишнее пробуждение

Первое решение lockSlow: можно ли активно подождать?



Что такое активное ожидание

- G остается RUNNING и проверяет лок, не отдавая P.
- Цена: M/P/CPU заняты почти бесполезной работой.
- Выгода: на короткой критической секции парковка может быть дороже.

Когда активное ожидание оправдано

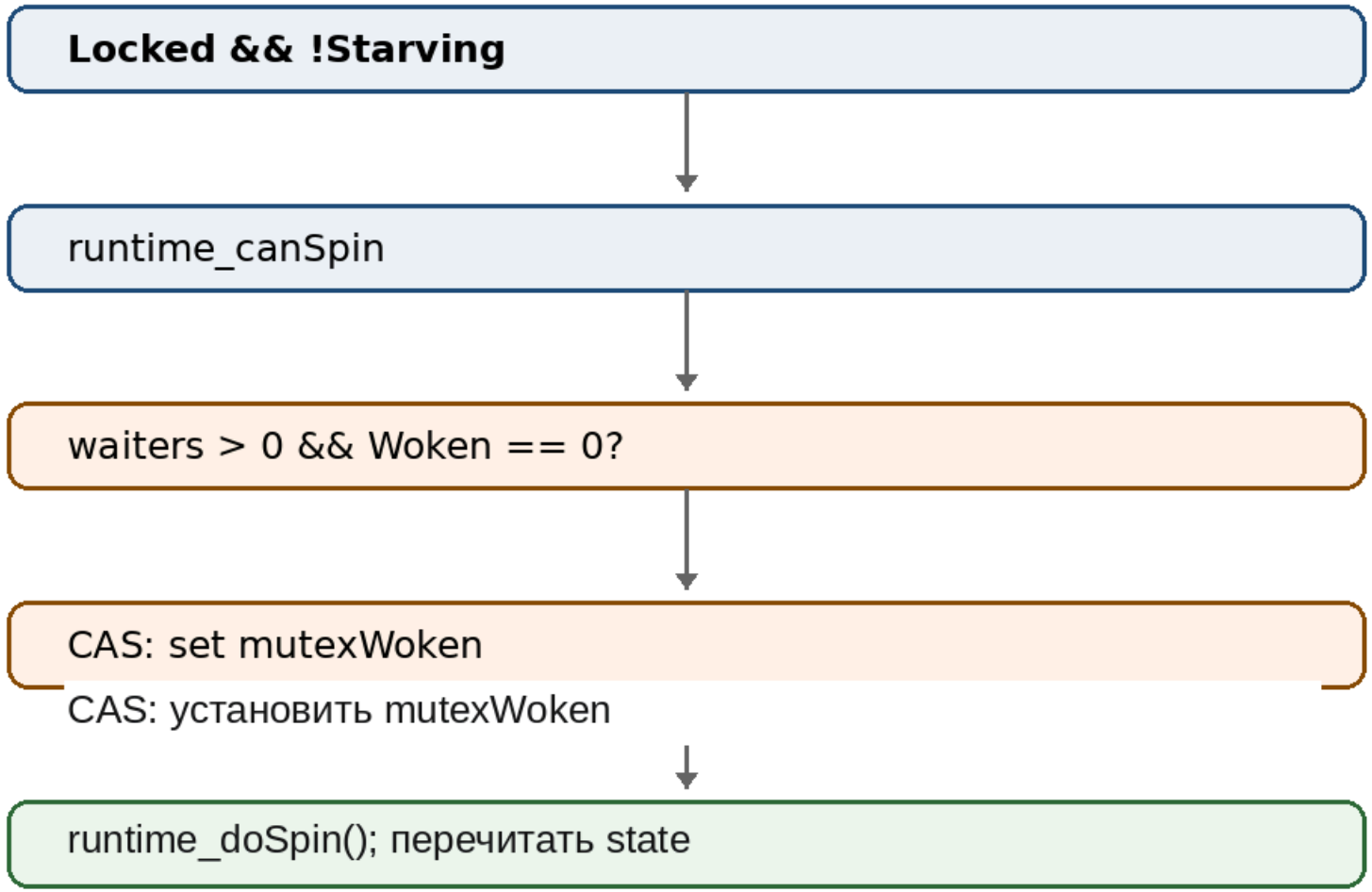
Хороший случай

- несколько CPU/P
- другая G может освободить лок
- локальная очередь текущего P пуста
- ожидание будет коротким

Плохой случай

- GOMAXPROCS=1
- есть готовые G
- критическая секция длится миллисекунды
- режим голодания уже активен

Активное ожидание и mutexWoken



После активного ожидания: строим новый state

- В обычном режиме G может запросить mutexLocked.
- Если лок занят/голодает: увеличить число ожидающих (+1 << 3).
- Если G ждет слишком долго: выставить mutexStarving.
- Если awoke=true: снять глобальный флаг mutexWoken.

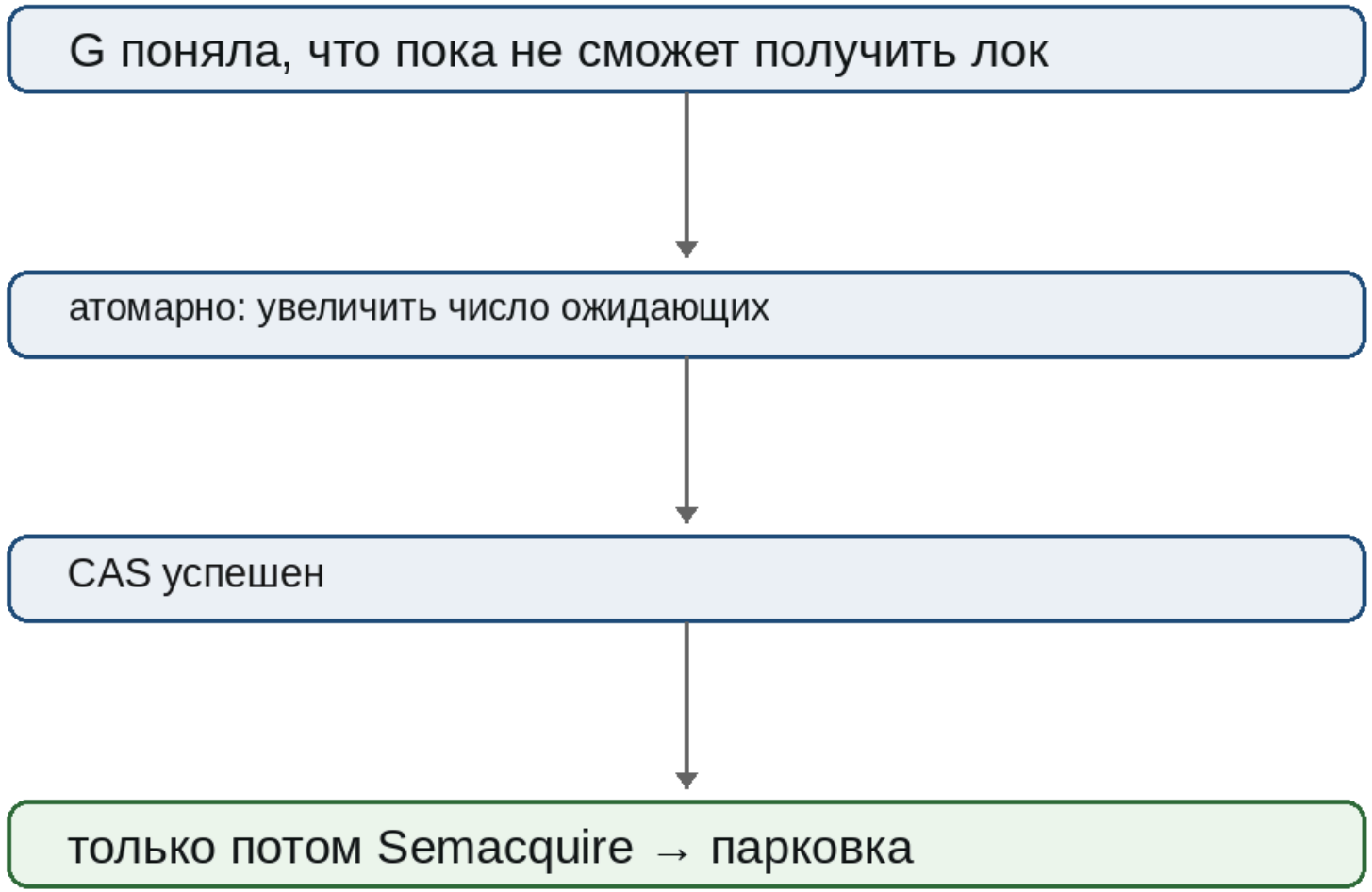
Число ожидающих хранится в старших битах

```
new += 1 << mutexWaiterShift

// mutexWaiterShift == 3
// +8 к state означает еще одну ожидающую G
```

Например: занят + 2 ожидающие G = 1 + 16 = 17.

Почему число ожидающих растет до парковки



Иначе Unlock увидит 0 ожидающих и потеряет пробуждение.

Голодание: скорость против справедливости

- В обычном режиме пробужденная G конкурирует с новыми горутинами.
- Новая G уже RUNNING и может выиграть CAS — перехват лока.
- После долгого ожидания Mutex переходит в режим голодания.
- Порог порядка 1 мс — деталь реализации, не гарантия API.

Обычный режим и режим голодания

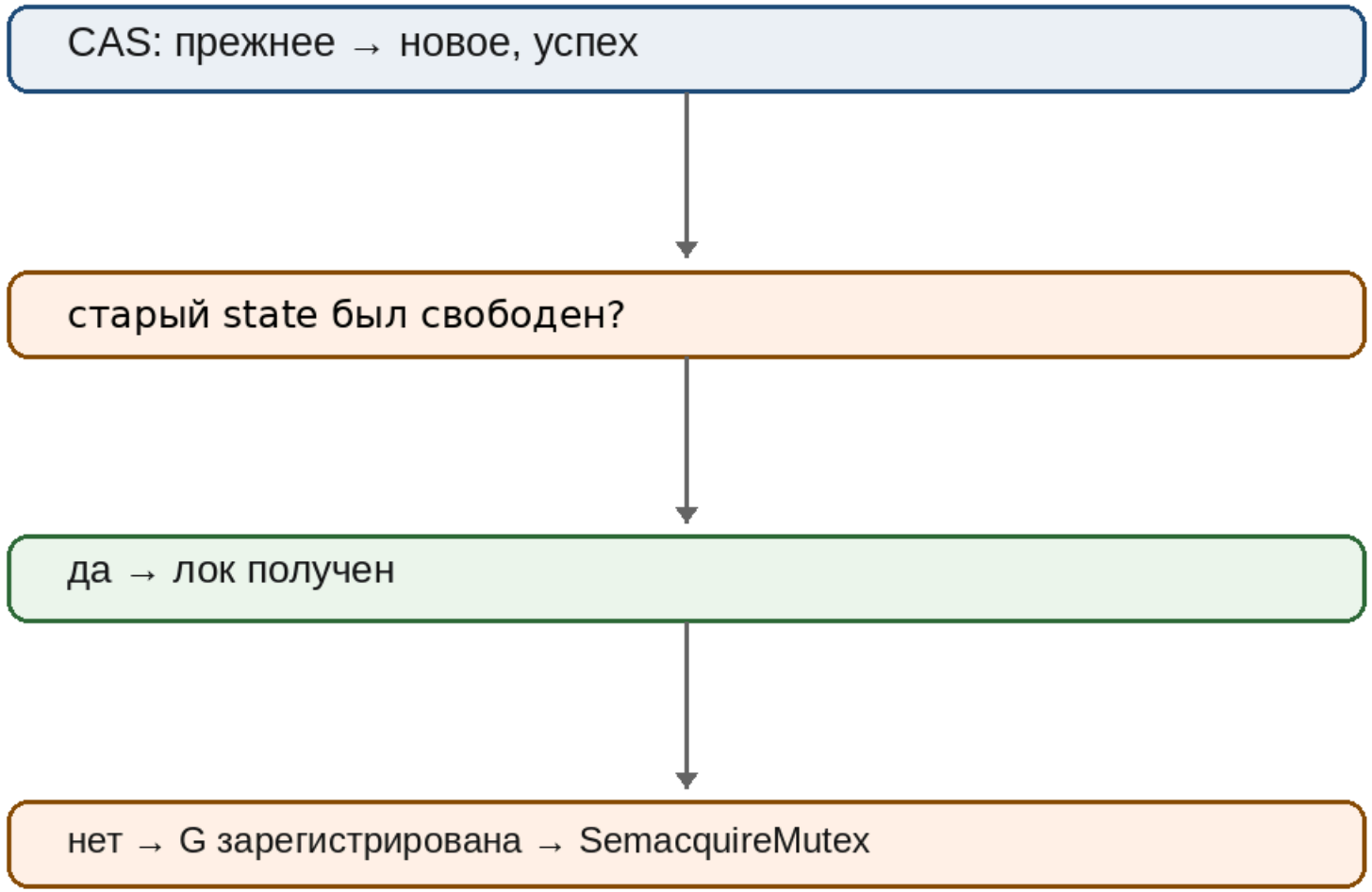
Обычный режим

- Unlock будит ожидающую G
- G снова конкурирует за лок
- новая G может перехватить лок
- выше пропускная способность

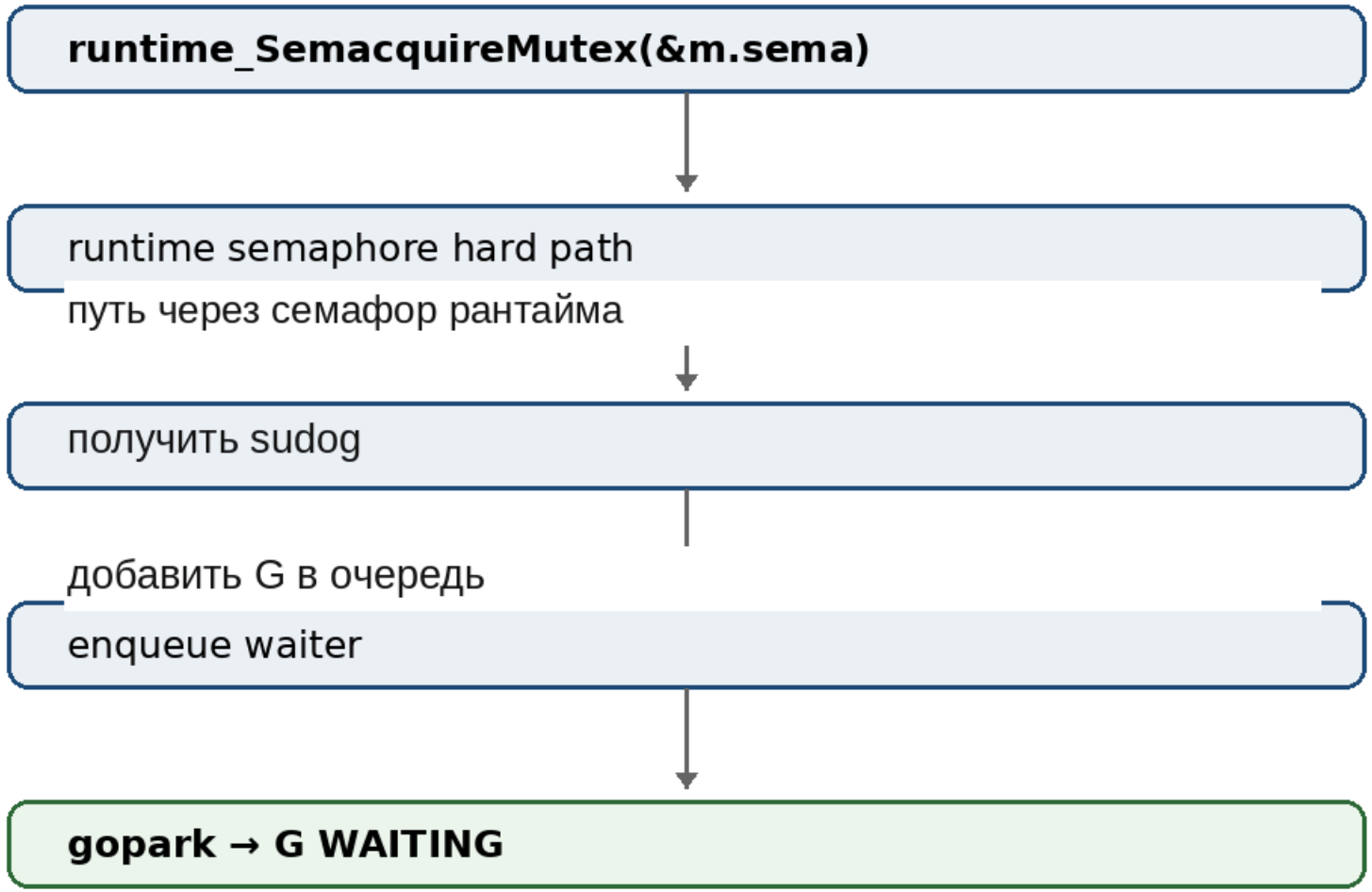
Голодание

- очередь получает приоритет
- новые G не перехватывают лок
- лок передается следующей G
- меньше хвостовых задержек

CAS успешен: два сценария



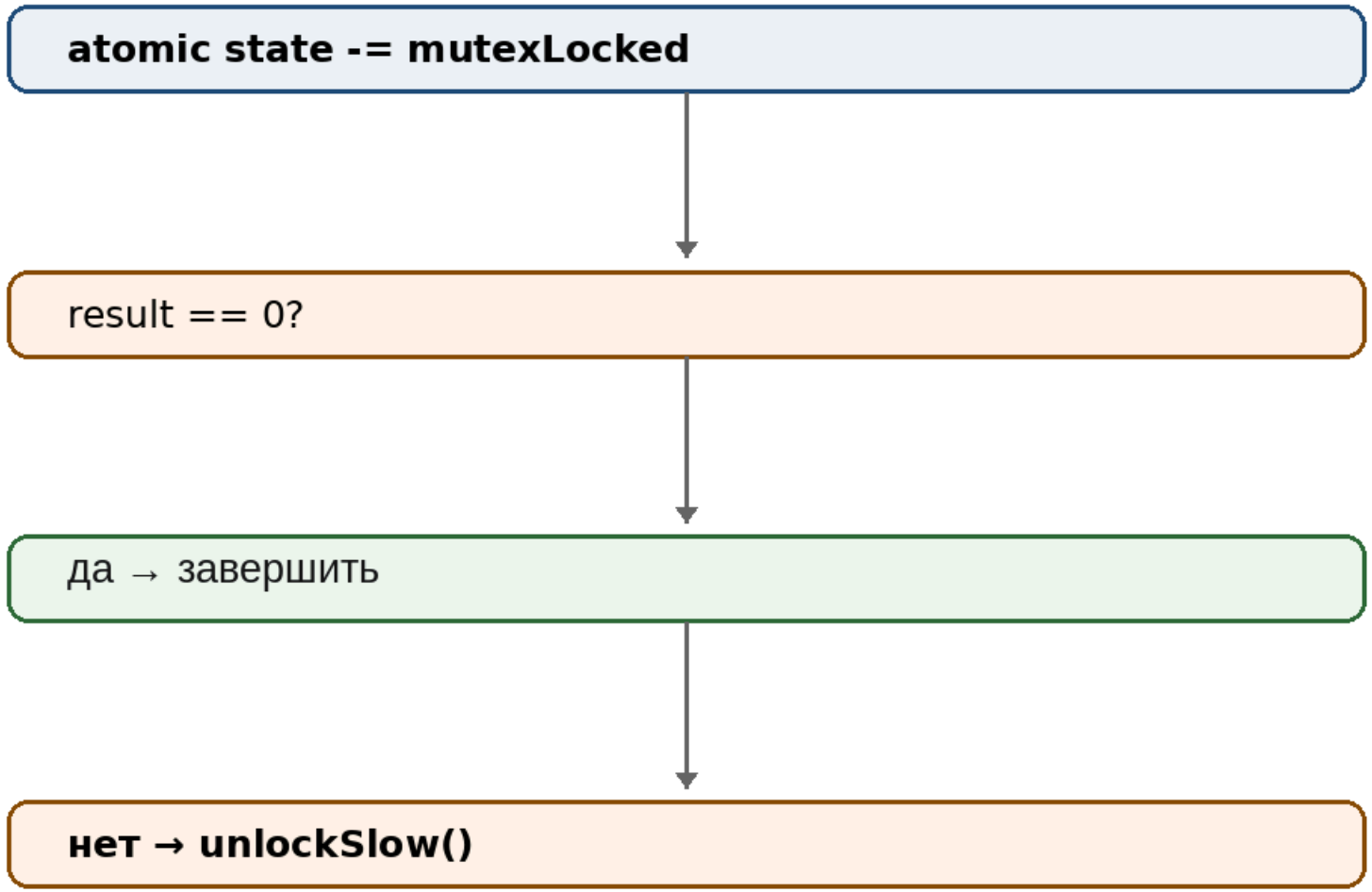
Медленный путь с ожиданием



Что происходит с M/P во время ожидания

- Ожидающая G переходит из RUNNING в WAITING.
- M больше не обязан оставаться связанным с этой G.
- Планировщик на том же P может выполнять другую готовую G.
- Конкурентный Go Mutex блокирует горутину, но не выделяет ей поток ОС.

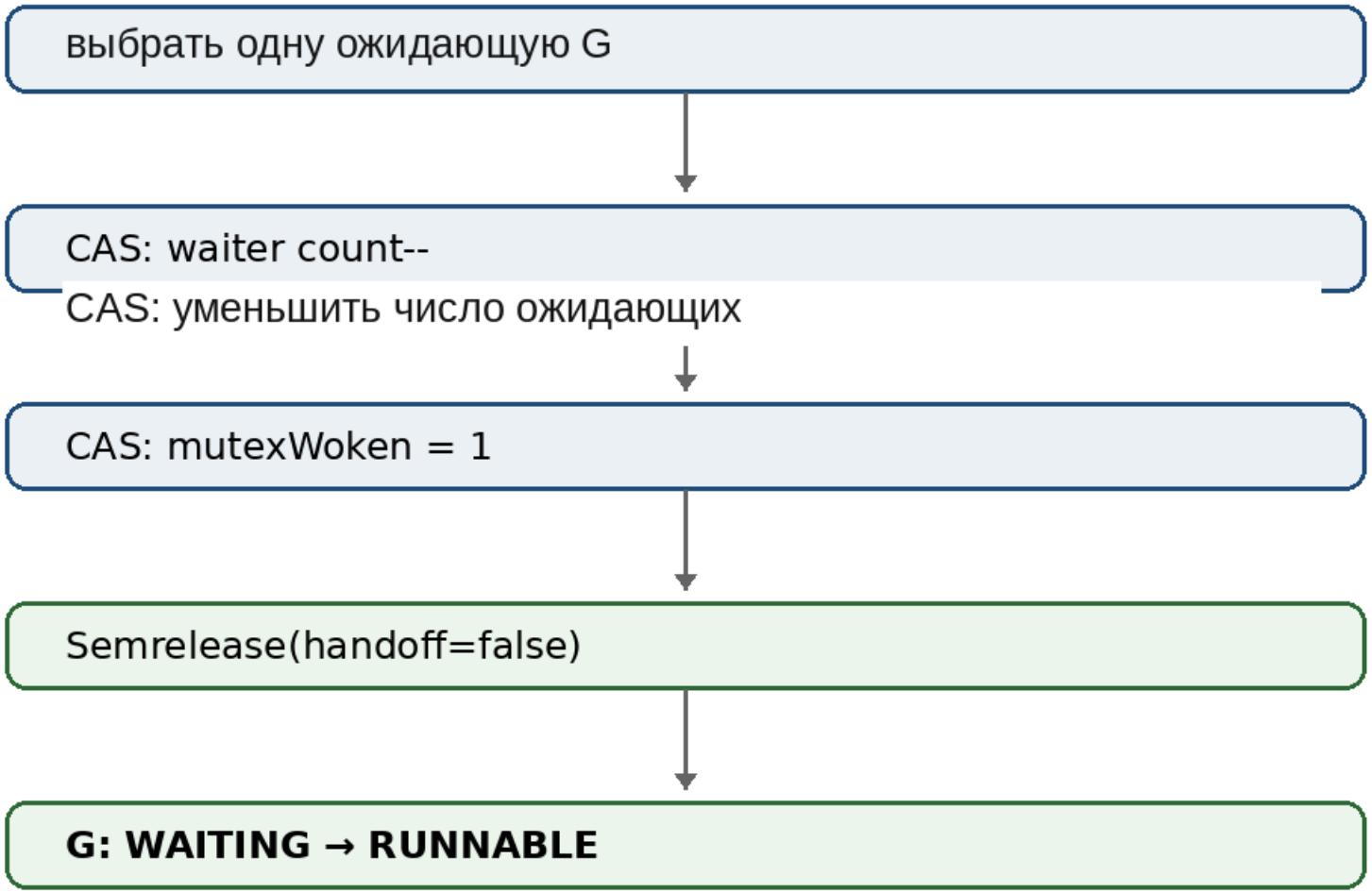
Unlock(): быстрый путь тоже короткий



Обычный unlockSlow: когда не будим G

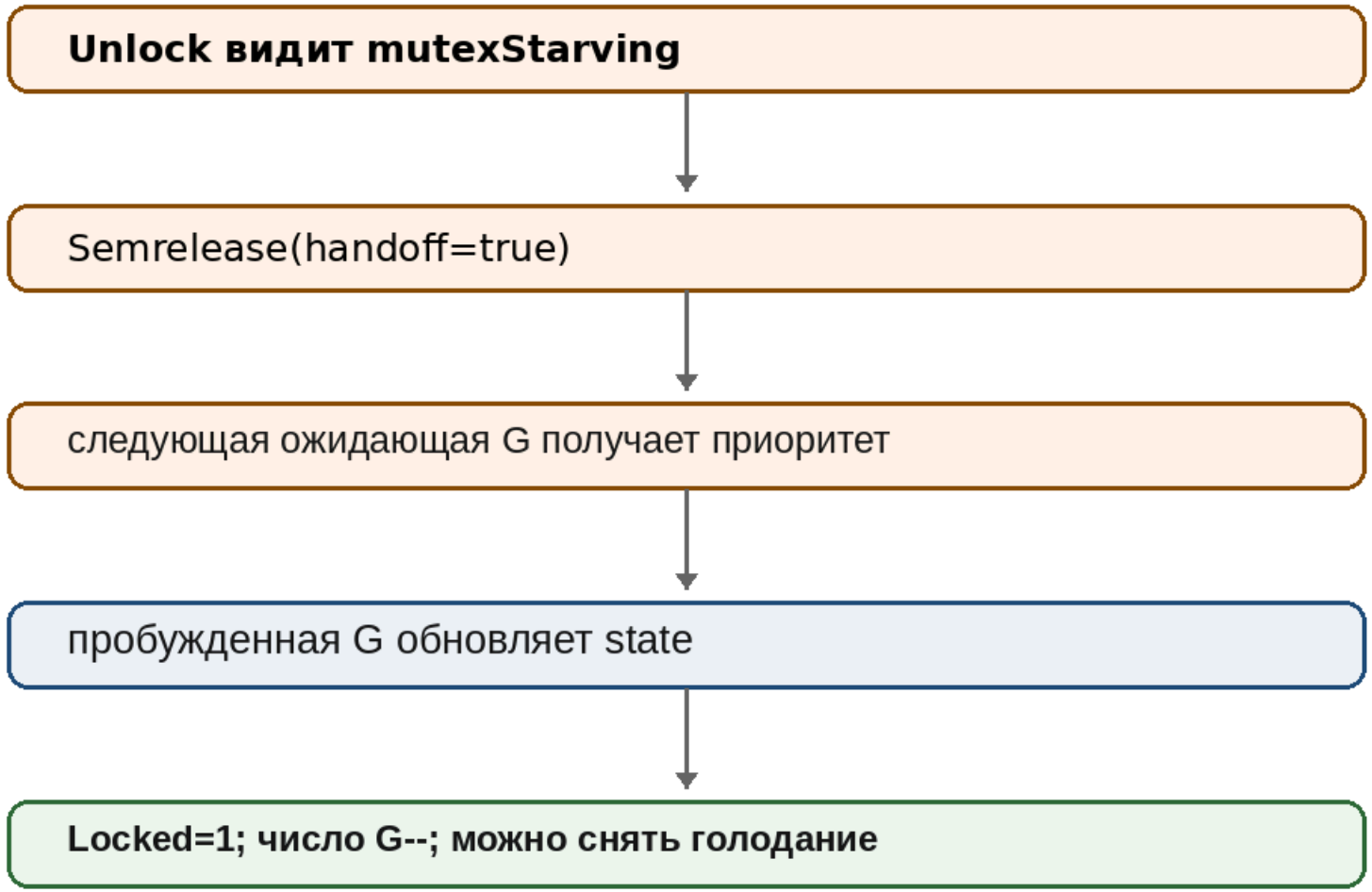
- waiters == 0 — будить некого.
- Mutex снова занят — другой претендент успел его захватить.
- mutexWoken == 1 — один активный кандидат уже есть.
- mutexStarving == 1 — используется протокол передачи владения.

Протокол пробуждения в обычном режиме



Сначала фиксируем состояние, затем реально будим G.

Передача владения при голодании

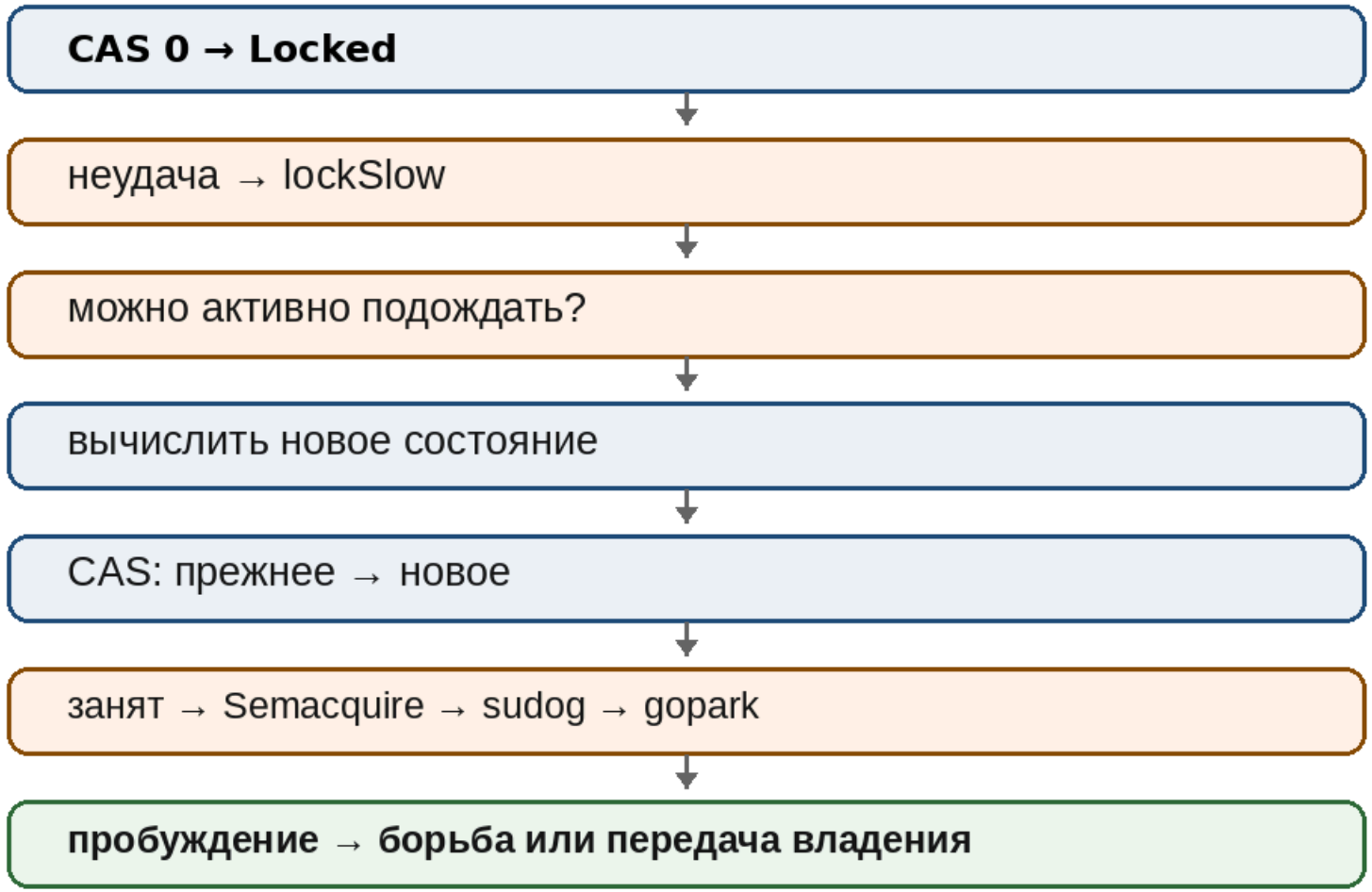


Почему при голодании встречается Locked = 0

```
state:
    mutexStarving = 1
    mutexLocked   = 0
    waiters       > 0
```

Здесь Locked=0 не значит свободу: mutexStarving закрепляет владение за очередью.

Полный путь Lock



Что меняется под нагрузкой

Малая конкуренция

- почти все Lock = один CAS
- минимум работы планировщика
- Mutex выглядит «бесплатным»

Высокая конкуренция

- неудачные CAS
- активное ожидание
- очереди ожидания
- парковка / пробуждение
- очередь и задержка p99

Антипаттерн: держать Mutex во время I/O

- Lock → БД/HTTP/диск/ожидание → Unlock создает очередь
  перед медленной зависимостью.
- CPU может быть загружен слабо, хотя задержка огромна.
- Узкое место — последовательный доступ и циклы парковки, не CAS.

Диагностика

- Профиль Mutex показывает, где время уходит на конкурентные локи.
- Профиль блокировок показывает реальные ожидания горутин.
- go tool trace: RUNNING → WAITING → RUNNABLE → RUNNING.
- Профиль CPU отделяет активное ожидание от полезной работы.
- Вопрос: как долго удерживается лок и что выполняется внутри критической секции?

Что важно запомнить

- state — упакованный атомарный автомат состояний.
- Быстрый путь — CAS, медленный — адаптивный протокол.
- mutexWoken предотвращает лишние пробуждения.
- Число ожидающих увеличивается до парковки, чтобы не потерять сигнал.
- Активное ожидание расходует CPU ради меньшей задержки.
- Обычный режим ускоряет работу; голодание повышает справедливость.
- Под нагрузкой цена Mutex — очередь, а не атомарная инструкция.